

Explanation for the Sentivity Analysis Module Code

Yunru Zheng

January 5, 2026

1 Introduction

All of the code is put in the "SensitivityAnalysisModule" folder, which is further divided into 3 folders. "src" is the source code where I put all the functions I wrote. Most of the source code is in "model.py", where, for example, I put the score function and the gradient descent function. I put the body of the Robbins Monro algorithm in "optimize.py". All the supplementary functions are put in "utils.py".

In tests, I tried to recreate the results of the examples in the notes I got. I named them from "ex1" to "ex5".

I converted the body of my previous "SensitivityAnalysisImproved2.py" into "main.py", which is stored inside "experiment". This folder is also where I put all the python files that use the source code.

2 Replicating the Example Results

I generated similar results to the example section in the notes using my own code and functions, which shows that my functions are indeed generating the results as desired. I tried to formulate the output style similar to yours. σ is the twisting parameter, basically "r" in the notes. To be consistent with the notes, I chose X^4 as an example of X^{2n} .

Here are the results of my "text_ex1_variance.py". I did it for X^{2n} for n from 1 to 15 (I only showed the first 10 here), but if you check with the notes, for X^4 , the results are consistent with example 1 the notes.

```

For X^2 using n = 5.00e+07 samples and twisted standard deviation r = 1.41e+00
direct estimate is 1.0000e+00, with sample variance 1.9992e+00
direct estimate is 9.9987e-01, with sample variance 5.3964e-01

For X^4 using n = 5.00e+07 samples and twisted standard deviation r = 2.00e+00
direct estimate is 2.9999e+00, with sample variance 9.5925e+01
direct estimate is 2.9995e+00, with sample variance 7.9279e+00

For X^6 using n = 5.00e+07 samples and twisted standard deviation r = 2.45e+00
direct estimate is 1.5010e+01, with sample variance 1.0182e+04
direct estimate is 1.5004e+01, with sample variance 2.7022e+02

For X^8 using n = 5.00e+07 samples and twisted standard deviation r = 2.83e+00
direct estimate is 1.0503e+02, with sample variance 2.0294e+06
direct estimate is 1.0499e+02, with sample variance 1.6384e+04

For X^10 using n = 5.00e+07 samples and twisted standard deviation r = 3.16e+00
direct estimate is 9.4925e+02, with sample variance 6.9563e+08
direct estimate is 9.4501e+02, with sample variance 1.5569e+06

For X^12 using n = 5.00e+07 samples and twisted standard deviation r = 3.46e+00
direct estimate is 1.0298e+04, with sample variance 3.5484e+11
direct estimate is 1.0395e+04, with sample variance 2.1388e+08

For X^14 using n = 5.00e+07 samples and twisted standard deviation r = 3.74e+00
direct estimate is 1.3310e+05, with sample variance 1.7977e+14
direct estimate is 1.3513e+05, with sample variance 4.0152e+10

For X^16 using n = 5.00e+07 samples and twisted standard deviation r = 4.00e+00
direct estimate is 2.0702e+06, with sample variance 1.0803e+17
direct estimate is 2.0269e+06, with sample variance 9.8739e+12

For X^18 using n = 5.00e+07 samples and twisted standard deviation r = 4.24e+00
direct estimate is 3.5541e+07, with sample variance 1.2354e+20
direct estimate is 3.4467e+07, with sample variance 3.0841e+15

For X^20 using n = 5.00e+07 samples and twisted standard deviation r = 4.47e+00
direct estimate is 5.7252e+08, with sample variance 7.9903e+22
direct estimate is 6.5465e+08, with sample variance 1.1914e+18

```

Figure 1: Importance Sampling and variance reduction for X^{2n} .

Here are the results of my "text_ex2_score_function_sensitivity.py". The numbers are very similar to the output in the notes. I am showing that for X^4 , the expectation $E[X^4]$ is 3, and the derivative of that is 12. This is also shown by decreasing the increments, since the value converges to the calculated value as the increment decreases.

```
baseline u0 = u(sigma0) = 3.0007e+00, computed sentivity is 1.1996e+01
```

dsigma	u(sigma0+dsigma) (theory)	u(sigma0+dsigma) (computed)	du/dsigma
2.0e-01	6.2208e+00	6.2220e+00	1.6106e+01
1.0e-01	4.3923e+00	4.3951e+00	1.3944e+01
5.0e-02	3.6465e+00	3.6464e+00	1.2915e+01
2.0e-02	3.2473e+00	3.2459e+00	1.2261e+01
1.0e-02	3.1218e+00	3.1206e+00	1.1993e+01

Figure 2: Verifying that $\frac{du}{d\sigma}$ for the target function converges to the calculated value.

Here are the results of my "text_ex3_variance_sensitivity.py". Instead of the mean, I calculated the variance of X^4 and my "grad_m_function" (your sensitivity). I also showed that by decreasing the increments in σ , the "m.function" (your ssq) increases, and the sensitivity gets close to the calculated value.

```
baseline variance = 9.6061e+01, computed sentivity is -8.4165e+02
```

dsigma	ssq(sigma0+dsigma) (computed)	dssq/dsigma
1.0e-01	4.7227e+01	-4.8834e+02
1.0e-02	8.8202e+01	-7.8590e+02
5.0e-03	9.1991e+01	-8.1396e+02
2.0e-03	9.4258e+01	-9.0149e+02
1.0e-03	9.5321e+01	-7.4062e+02

Figure 3: Showing that when the increment is small, the derivative of ssq gets close to the computed sensitivity.

Here are the results of my "text_ex4_stochastic_gradient_descent.py". I am using the Robbins Monro algorithm to find the optimal sigma. In the following 4 graphs, I used different step size and batch size, and that results in different smoothness and convergence. However, you can see that in general they converge to the optimal sigma value.

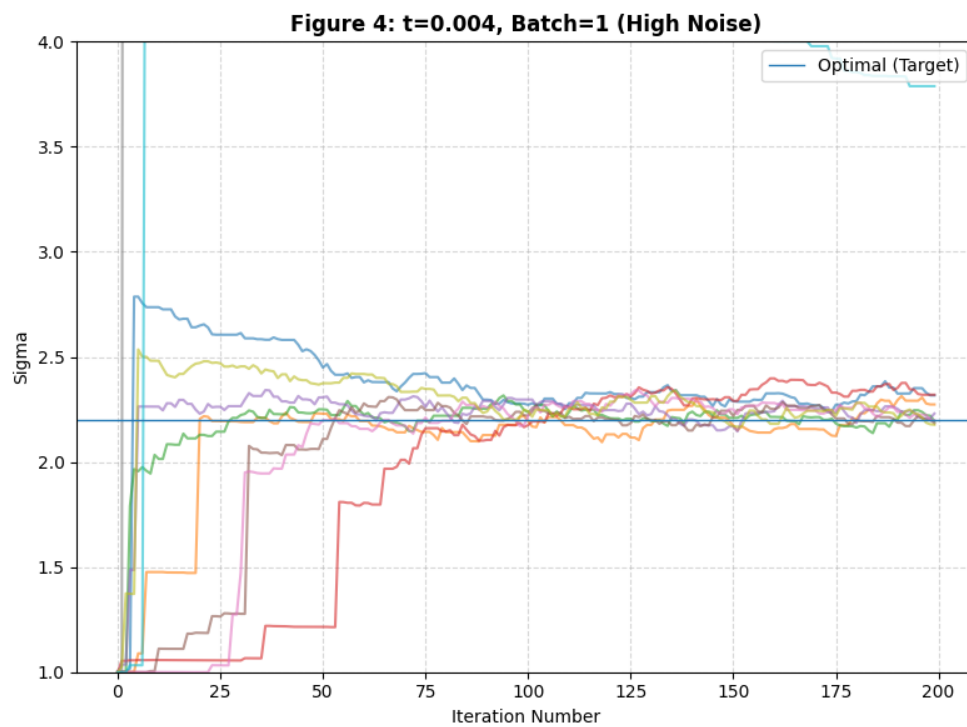


Figure 4: $t=0.004$, Batch=1 (High Noise).

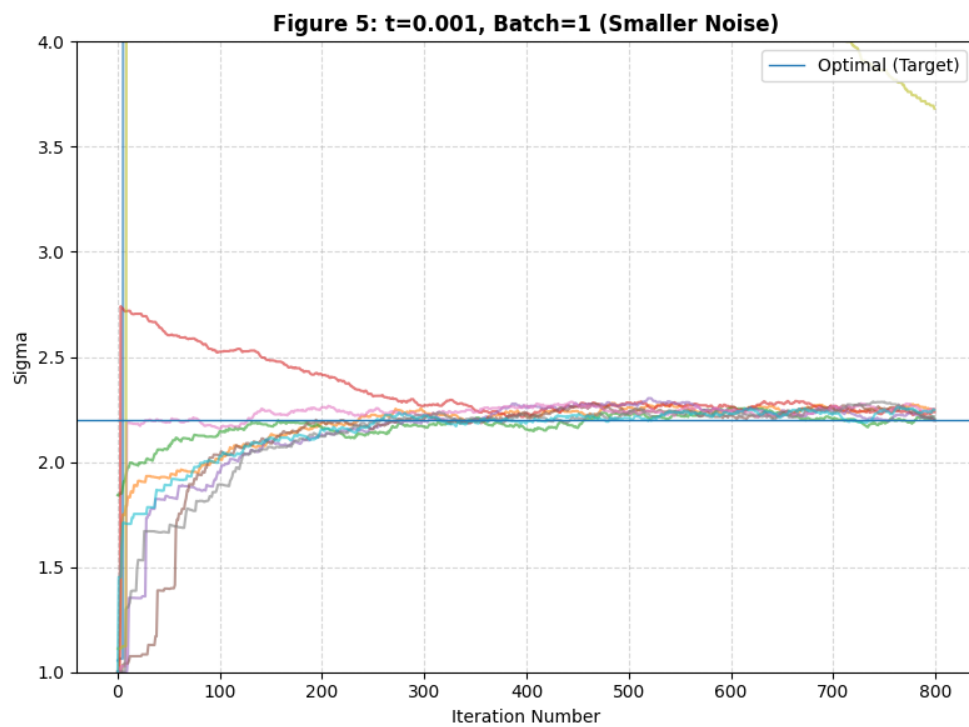


Figure 5: $t=0.001$, Batch=1 (Smaller Noise).

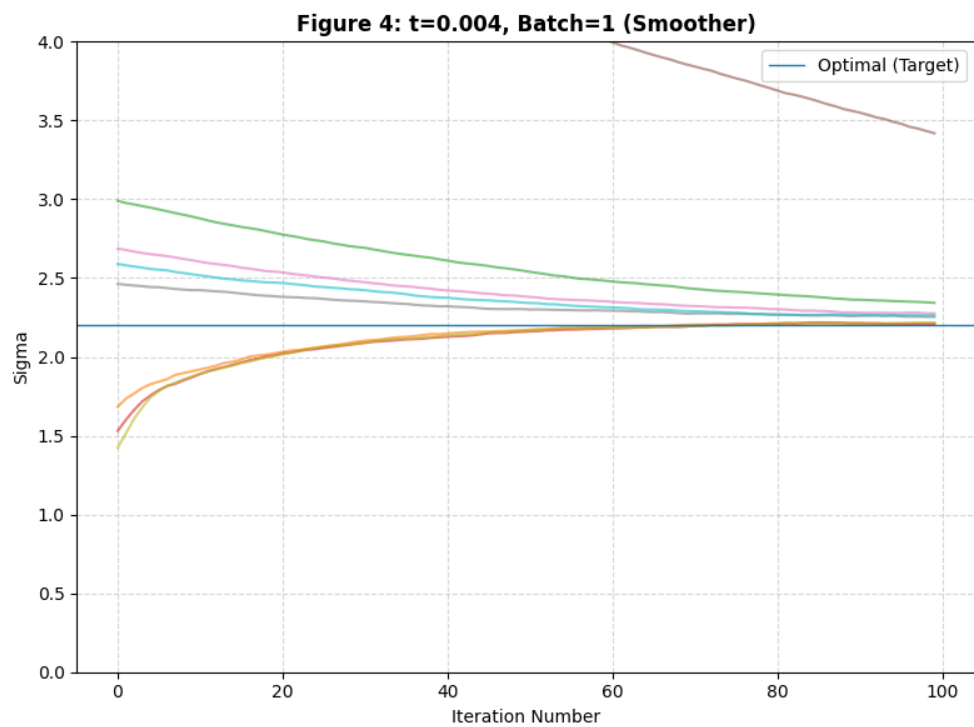


Figure 6: $t=0.004$, Batch=100 (Smoother).

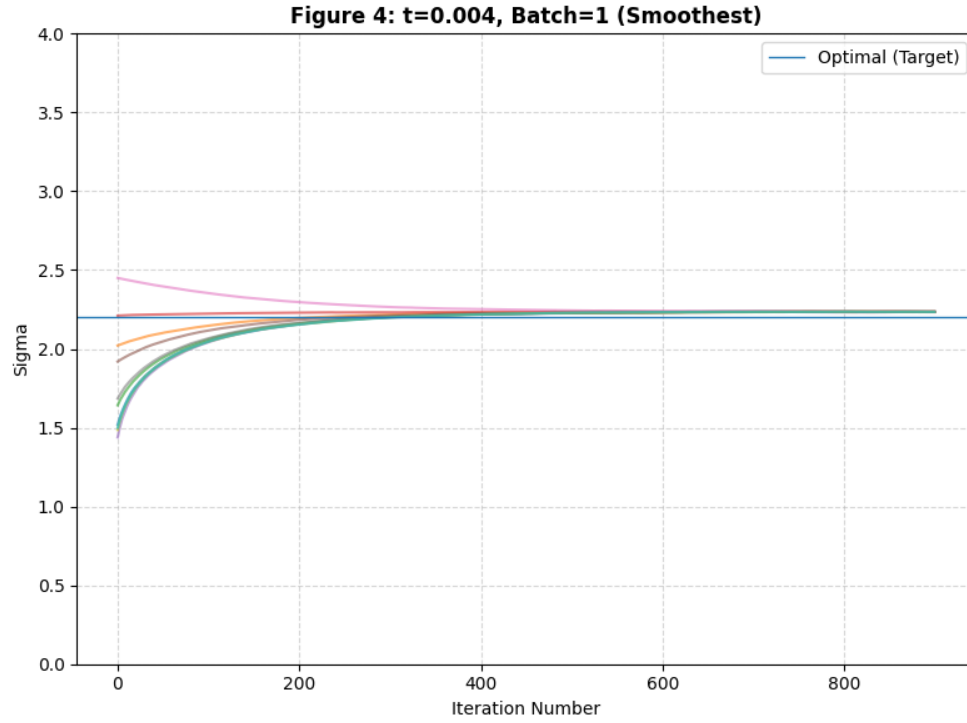


Figure 7: $t=0.001$, Batch=1000 (Smoothest).

Here are the results of my "text_ex5_estimation_variance.py". By calculating the variance corresponding to importance sampling using different sigma values, I found that the optimal sigma is 2.18, which is close to $\sigma^* \approx 2.2$.

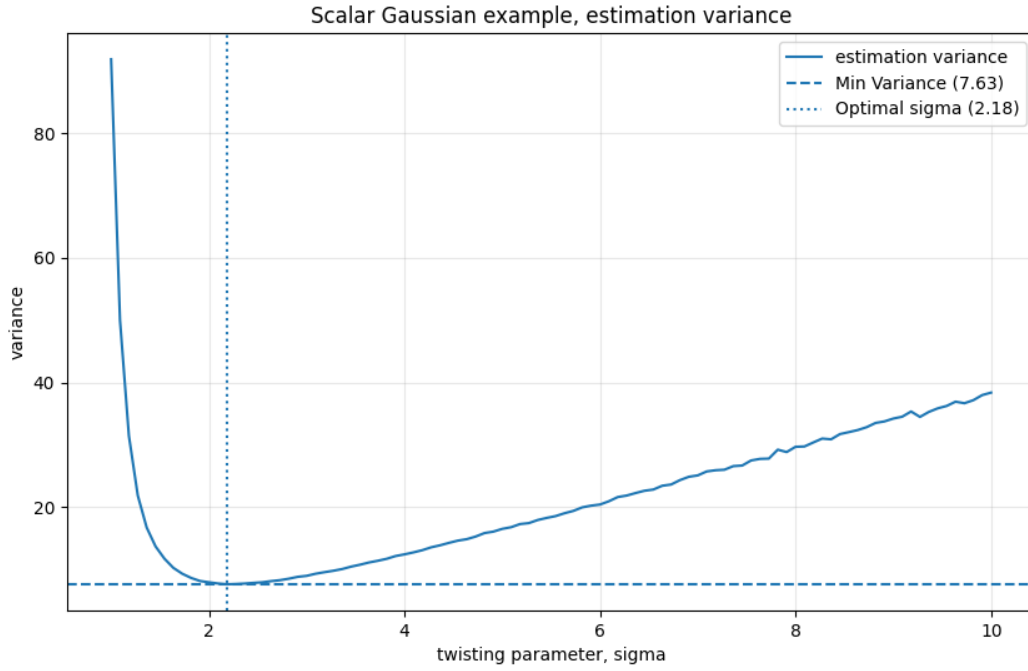


Figure 8: Variance as a function of sigma.

3 Batman

I put the code and graphs for batman graph as "batman.py" under "experiments". I tried to replicate the batman graph by trying different parameters $\theta = (a, b, c, d)$. I used $a = 2.0, b = 3.9, c = 1.0, d = 1.0$, which gives a nice graph for X as a function of Z , the standard Gaussian. However, the normal distribution seems to dominate the histogram and I am not able to create batman ears while the slope is positive.

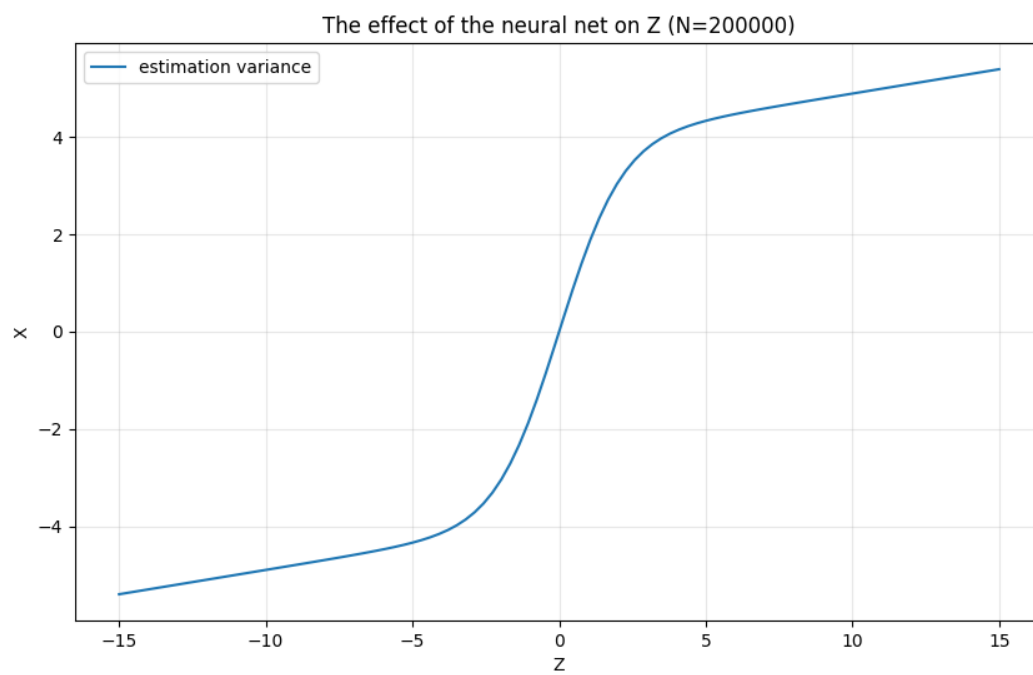


Figure 9: X vs Z as $a = 2.0, b = 3.9, c = 1.0, d = 1.0$.

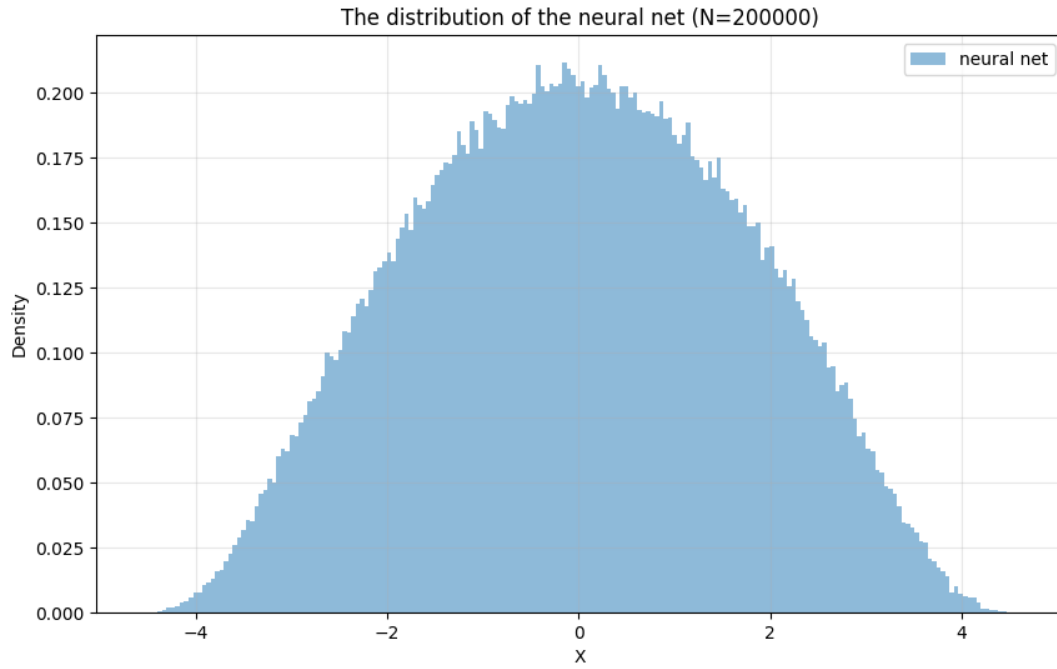


Figure 10: Distribution of X as $a = 2.0, b = 3.9, c = 1.0, d = 1.0$.

Later I changed my parameters to $a = 1.0, b = 3.9, c = 2.0, d = 1.0$, which makes the slope change from positive to negative. This finally gives me some ear of batman but I didn't see your original graph, so I'm not sure whether this is what you want. I'm feeling sketchy about this because X decreases as Z increases.

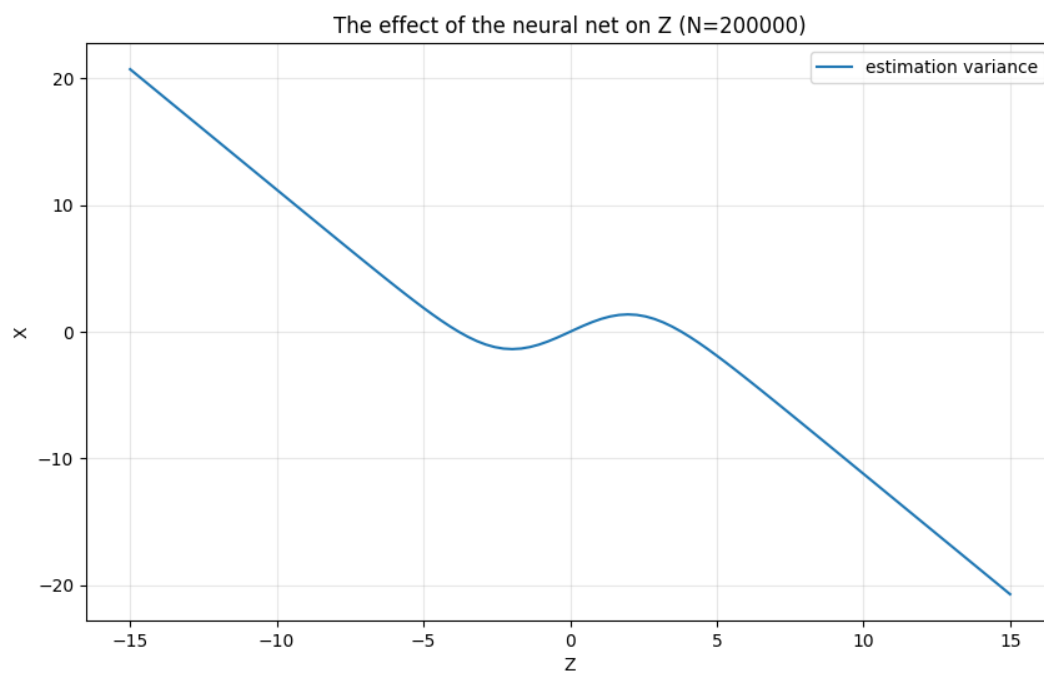


Figure 11: X vs Z as $a = 1.0, b = 3.9, c = 2.0, d = 1.0$.

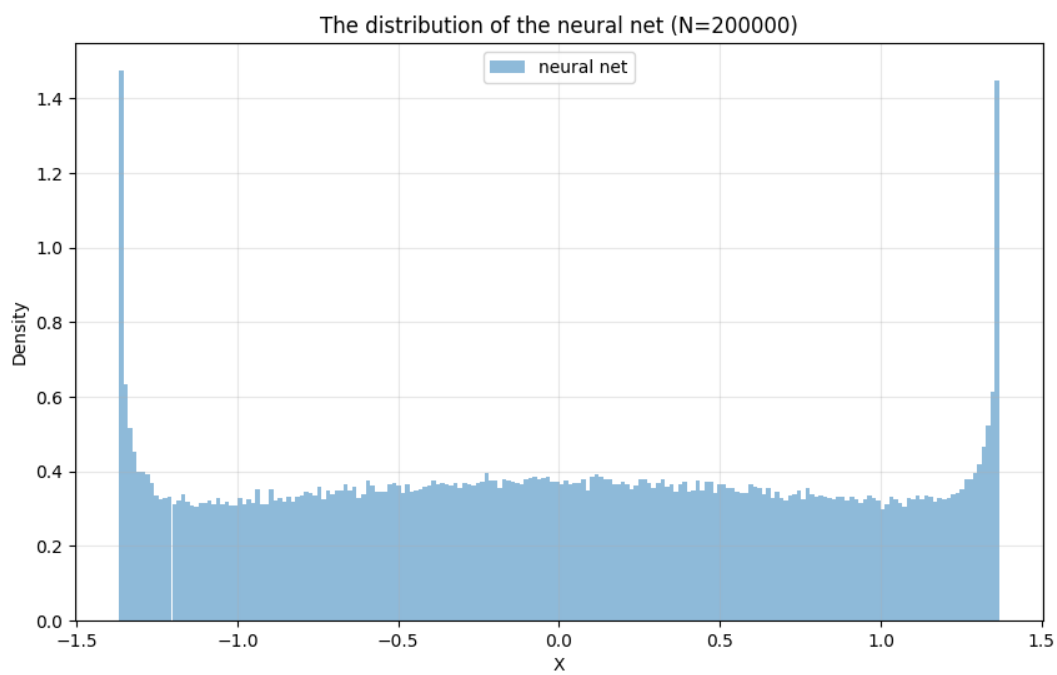


Figure 12: Distribution of $a = 1.0, b = 3.9, c = 2.0, d = 1.0$.